

Projeto Multidisciplinar Desenvolvimento Back-End

Aluna: Desyree N. Garcia Batista
RU: 986134
Curso: Análise e Desenvolvimento de Sistemas

Sumário

INTRODUÇÃO	3
ANÁLISE DE REQUISITOS:	4
Tabela de Requisitos Funcionais:	4
Tabela de Requisitos Não Funcionais:	4
Diagrama Casos de Uso:	5
MODELAGEM E ARQUITETURA (BACK-END)	6
Tecnologias Utilizadas:	6
Endpoints Principais da API:	6
Diagrama Entidade Relacionamento	6
Diagrama de Classes:	7
IMPLEMENTAÇÃO E PROTOTIPAGEM	8
Link do Repositório:	8
Estrutura de Pastas:	8
Prints Código:	8
Prints Swagger:	10
PLANO DE TESTES:	11
CONCLUSÃO:	14

INTRODUÇÃO

Este projeto apresenta o desenvolvimento do backend para o Sistema de Gestão Hospitalar e de Serviços de Saúde (SGHSS) da instituição VidaPlus. O objetivo central é a modernização e centralização de processos como cadastros de pacientes, agendamentos de consultas e gestão de prontuários, garantindo segurança de dados e alta disponibilidade.

ANÁLISE DE REQUISITOS:

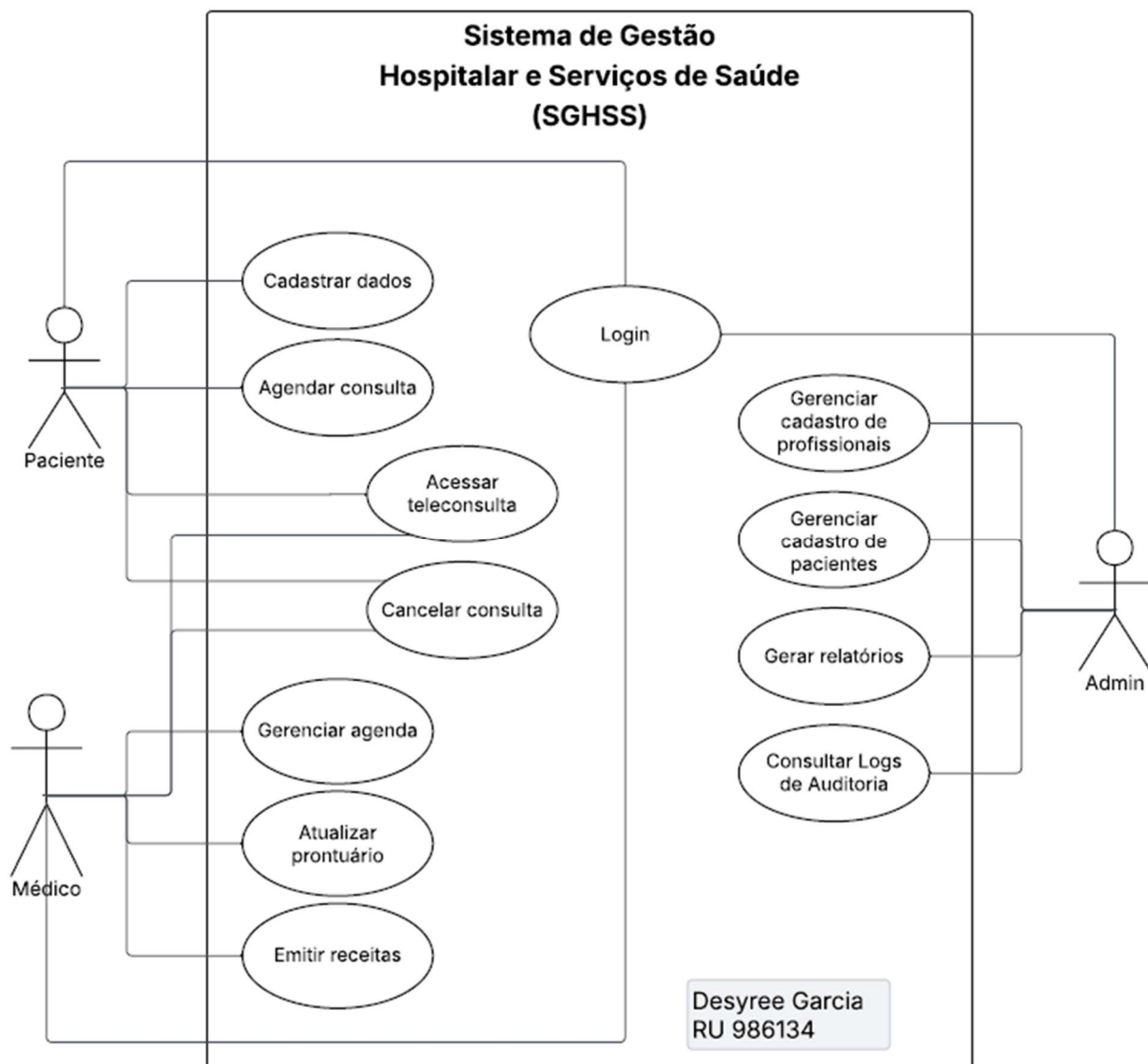
Tabela de Requisitos Funcionais:

ID	Requisito Funcional	Descrição	Ator Relacionado
RF01	Autenticação (Login)	O sistema deve validar as credenciais de acesso dos usuários.	Todos
RF02	Cadastrar Dados	Permite que o paciente insira suas informações pessoais no sistema.	Paciente
RF03	Agendar Consulta	Funcionalidade para marcar atendimentos presenciais ou por telemedicina.	Paciente
RF04	Cancelar Consulta	Permite ao paciente desistir de um horário previamente agendado.	Paciente
RF05	Acessar Teleconsulta	Interface segura para realização de chamadas de vídeo médicas.	Paciente
RF06	Gerenciar Agenda	O médico pode organizar seus horários e visualizar consultas marcadas.	Médico
RF07	Atualizar Prontuário	Registro das informações clínicas e histórico de saúde do paciente.	Médico
RF08	Emitir Receitas	Geração de prescrições digitais para medicamentos e exames.	Médico
RF09	Gerenciar Profissionais	Cadastro e manutenção de dados de médicos e técnicos.	Admin
RF10	Gerenciar Pacientes	Controle administrativo sobre a base de dados de todos os pacientes.	Admin
RF11	Gerar Relatórios	Emissão de documentos gerenciais e financeiros do hospital.	Admin
RF12	Consultar Logs	Acesso ao registro de atividades para fins de auditoria e segurança.	Admin

Tabela de Requisitos Não Funcionais:

ID	Requisito Não Funcional	Descrição
RNF01	Segurança (LGPD)	Garantir a criptografia de dados sensíveis e conformidade com a lei.
RNF02	Disponibilidade	O sistema deve estar disponível em no mínimo 99,5% do tempo.
RNF03	Desempenho	Tempo de resposta rápido, especialmente em consultas críticas.
RNF04	Escalabilidade	Capacidade de suportar múltiplas unidades hospitalares simultaneamente.
RNF05	Acessibilidade	Interface amigável e responsiva seguindo padrões W3C/WCAG.

Diagrama Casos de Uso:



MODELAGEM E ARQUITETURA (BACK-END)

Tecnologias Utilizadas:

- Python 3.11+: Linguagem base do projeto.
- FastAPI: Framework moderno de alta performance para construção de APIs.
- SQLAlchemy: ORM (Object Relational Mapper) para manipulação simplificada do banco de dados.
- SQLite: Banco de dados relacional leve para armazenamento local.
- Pydantic v2: Validação rigorosa de dados e tipagem de contratos (Schemas).
- Docker: Containerização para facilitar o deploy e padronização do ambiente.

Endpoints Principais da API:

- GET /pacientes: Lista todos os pacientes.
- GET /consultas/medico/{medico_id}: Permite que o médico visualize sua agenda específica do dia.
- POST /consultas: Agenda um novo atendimento.

Diagrama Entidade Relacionamento

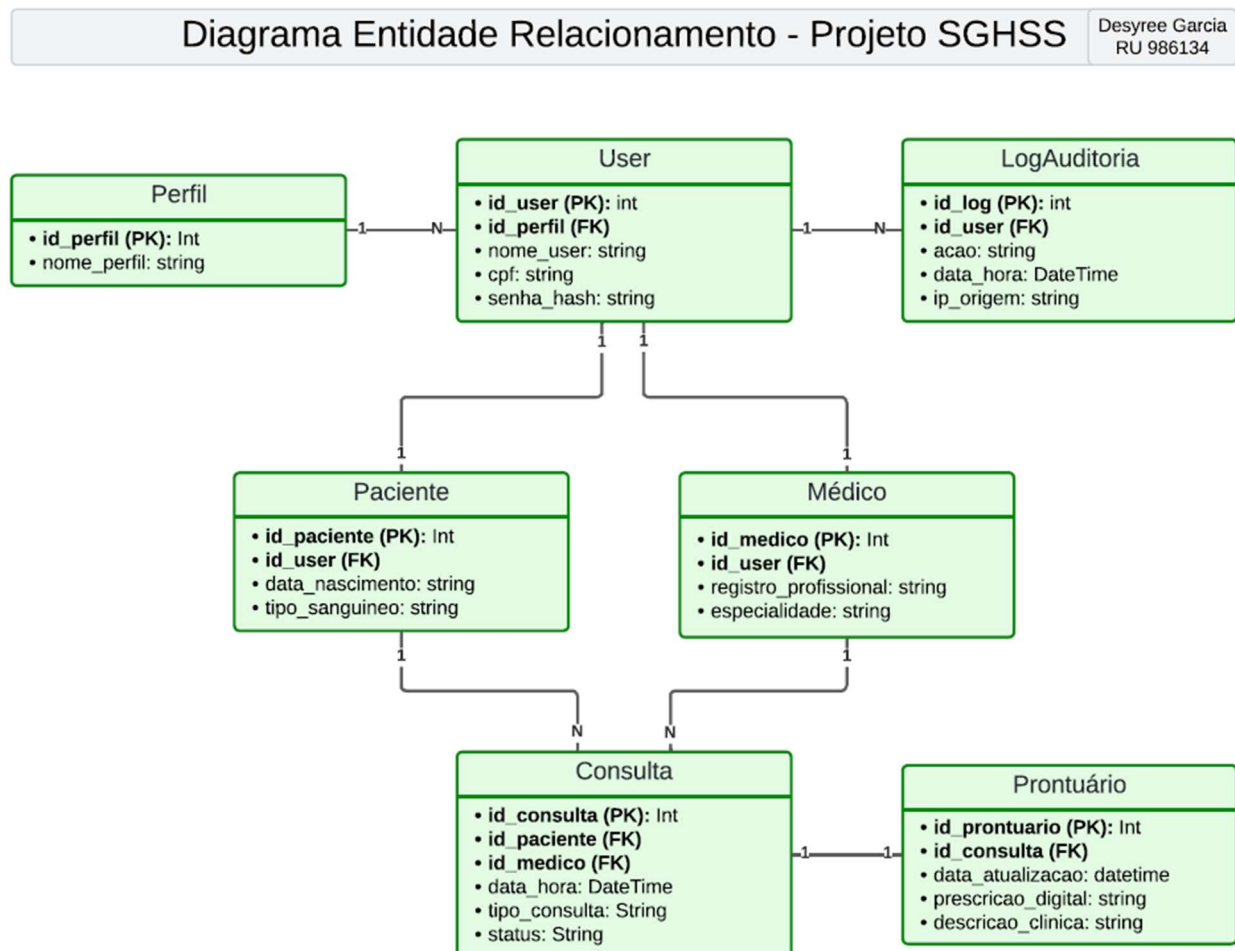
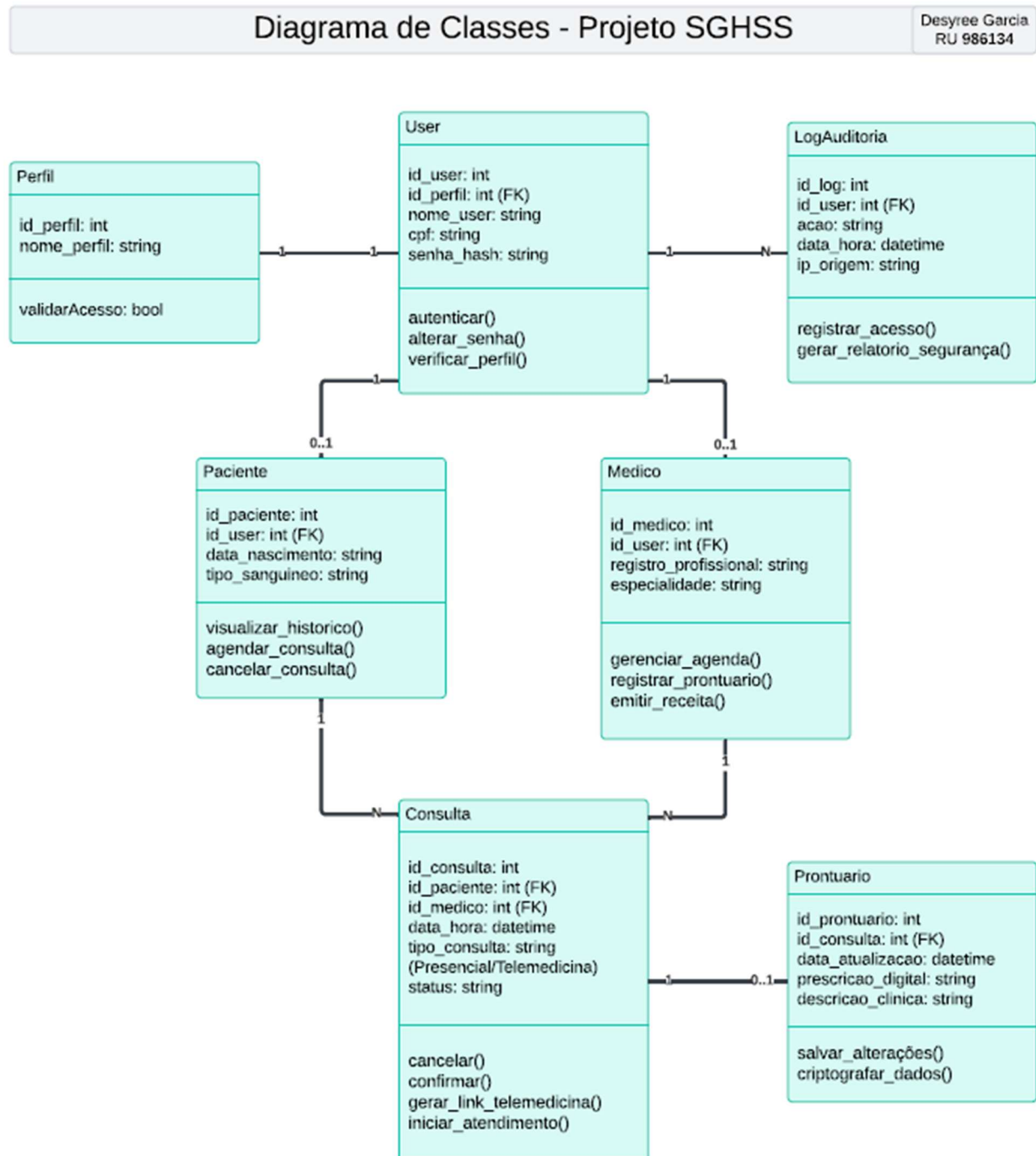


Diagrama de Classes:



IMPLEMENTAÇÃO E PROTOTIPAGEM

Link do Repositório:

<https://github.com/desyreegarcia/Projeto-BackEnd-SGHSS.git>

Estrutura de Pastas:

```
src/  
├── main.py  
├── models/  
│   ├── database.py  
│   ├── models.py  
│   └── schemas.py  
└── router/  
    ├── perfil.py  
    ├── user.py  
    ├── paciente.py  
    ├── medicos.py  
    └── atendimentos.py
```

Prints Código:

Modelagem da Base de Dados com SQLAlchemy:

```
src > models > models.py > ...  
1 # Importações e bibliotecas  
2 from sqlalchemy import Column, Integer, String,  
3 from sqlalchemy.orm import relationship # Função  
4 from .database import Base # Base que foi criada  
5 import datetime # Para manipular datas e horas  
6  
7 # Tabela de Perfil de acesso  
8 class Perfil(Base):  
9     __tablename__ = "perfil" # Nome da tabela no banco  
10     id_perfil = Column(Integer, primary_key=True, index=True)  
11     nome_perfil = Column(String) # (i = 'Admin', 'Medico', 'Paciente')  
12  
13 # --- Tabela de Usuarios ---  
14 class User(Base):  
15     __tablename__ = "user"  
16     id_user = Column(Integer, primary_key=True, index=True)  
17     id_perfil = Column(Integer, ForeignKey("perfil.id_perfil"))  
18     nome_user = Column(String)  
19     cpf = Column(String, unique=True) # CPF - não pode ser repetido  
20     senha_hash = Column(String) # Senha criptografada  
21     user_ativo = Column(Boolean, default=True)  
22  
23     perfil = relationship("Perfil") # Um usuário tem um perfil  
24  
25 # --- Tabela de Pacientes ---  
26 class Paciente(Base):  
27     __tablename__ = "pacientes"  
28     id_paciente = Column(Integer, primary_key=True, index=True)  
29     id_user = Column(Integer, ForeignKey("user.id_user"))  
30     data_nascimento = Column(String)  
31     tipo_sanguineo = Column(String)  
32  
33     user = relationship("User") # Um paciente é um usuário  
34  
35 # --- Tabela de Médicos/Profissionais de Saúde ---  
36 class Medico(Base):  
37     __tablename__ = "medicos"  
38     id_medico = Column(Integer, primary_key=True, index=True)  
39     id_user = Column(Integer, ForeignKey("user.id_user"))  
40     registro_profissional = Column(String, unique=True)  
41     especialidade = Column(String)  
42  
43     user = relationship("User") # Um médico é um usuário  
44  
45 # --- Tabela de Consultas ---  
46 class Consulta(Base):  
47     __tablename__ = "consulta"  
48     id_consulta = Column(Integer, primary_key=True, index=True)  
49     id_paciente = Column(Integer, ForeignKey("pacientes.id_paciente"))  
50     id_medico = Column(Integer, ForeignKey("medicos.id_medico"))  
51     data_consulta = Column(String)  
52     hora_consulta = Column(String)  
53     tipo_consulta = Column(String) # Presencial ou Remota  
54     status = Column(String) # "Agendada", "Cancelada", "Concluída"  
55  
56 # Objetos relacionados  
57 paciente = relationship("Paciente")  
58 medico = relationship("Medico")  
59 prontuario = relationship("Prontuarios", back_populates="consulta")  
60  
61 # --- Tabela de Prontuario ---  
62 class Prontuarios(Base):  
63     __tablename__ = "prontuario"  
64     id_prontuario = Column(Integer, primary_key=True, index=True)  
65     id_consulta = Column(Integer, ForeignKey("consulta.id_consulta"))  
66     data_atualizacao = Column(DateTime, default=datetime.utcnow())  
67     prescricao_digital = Column(String)  
68     descricao_clinica = Column(String)  
69  
70     consulta = relationship("Consulta", back_populates="prontuario")  
71  
72 # --- Tabela de Auditoria (Conformidade com LGPD) ---  
73 class LogAuditoria(Base):  
74     __tablename__ = "logs_auditoria"  
75     id_log = Column(Integer, primary_key=True, index=True)  
76     id_user = Column(Integer, ForeignKey("user.id_user"))  
77     data_acao = Column(String) # Ex: "Acesso ao Prontuario"  
78     data_hora = Column(DateTime, default=datetime.utcnow())  
79     ip_origem = Column(String) # Registro de onde veio o acesso  
80  
81 # Objeto relacionado: permite acessar log.us  
82 user = relationship("User")  
83
```


Ponto de Entrada da Aplicação e Inicialização do FastAPI:

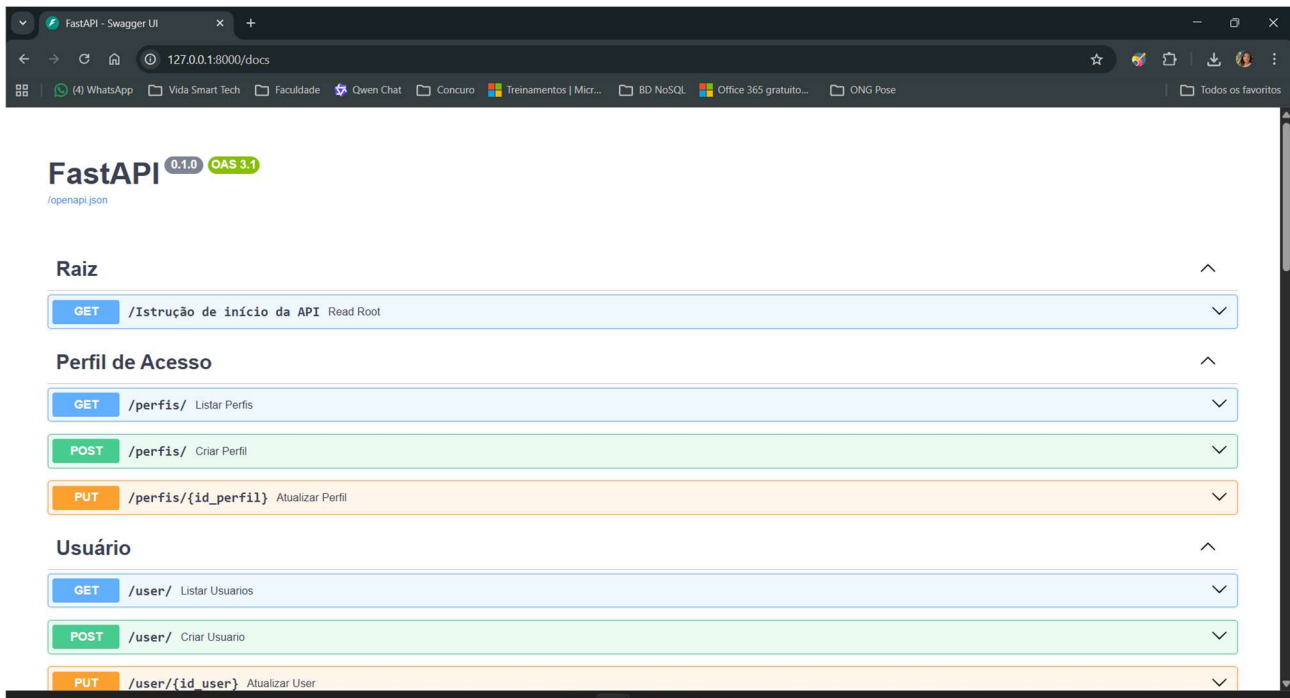
```
src > main.py > ...
1  from fastapi import APIRouter, FastAPI, Depends, HTTPException # Ferramentas do FastAPI
2  from sqlalchemy.orm import Session # Tipo de dado para a sessão do banco
3  from .models import models, schemas, database # Arquivos internos
4  from typing import List
5  from src.router import perfil, user, paciente, medicos, atendimentos
6
7  app = FastAPI()
8
9  # Mensagem de instrução para acessar a documentação no Swagger
10 @app.get("/Instrução de início da API", tags=["Raiz"])
11 def read_root():
12     return {"Mensagem: Para acessar a documentação, vá para /docs"}
13
14 # Comando que lê os modelos e cria as tabelas no arquivo sghss.db se elas não existirem
15 models.Base.metadata.create_all(bind=database.engine)
16
17 # Registro das rotas
18 app.include_router(perfil.router)
19 app.include_router(user.router)
20 app.include_router(paciente.router)
21 app.include_router(medicos.router)
22 app.include_router(atendimentos.router)
```

Definição de Rotas e Endpoints para Perfis de Acesso:

```
src > router > perfil.py > atualizar_perfil
1  #Imports
2  from fastapi import APIRouter, Depends, HTTPException # Ferramentas do FastAPI
3  from sqlalchemy.orm import Session # Tipo de dado para a sessão do banco
4  from ..models import models, schemas, database # Arquivos internos
5  from ..models.database import get_db
6  from typing import List
7
8  # Inicializa a aplicação FastAPI com título para o Swagger
9  router = APIRouter(prefix="/perfis", tags=["Perfil de Acesso"])
10
11 # Criar perfil de acesso
12 @router.post("/", response_model=schemas.Perfil)
13 def criar_perfil(perfil: schemas.PerfilCreate, db: Session = Depends(get_db)):
14     db_perfil = models.Perfil(nome_perfil=perfil.nome_perfil)
15     db.add(db_perfil)
16     db.commit()
17     db.refresh(db_perfil)
18     return db_perfil
19
20 # Listar perfis
21 @router.get("/", response_model=List[schemas.Perfil])
22 def listar_perfis(db: Session = Depends(get_db)):
23     return db.query(models.Perfil).all()
24
25 # Atualizar perfil buscando pelo id_perfil
26 @router.put("/{id_perfil}", response_model=schemas.Perfil)
27 def atualizar_perfil(id_perfil: int, perfil_atualizado: schemas.PerfilCreate, db: Session = Depends(get_db)):
28     db_perfil = db.query(models.Perfil).filter(models.Perfil.id_perfil == id_perfil).first()
29     db_perfil.nome_perfil = perfil_atualizado.nome_perfil
30     db.commit()
31     db.refresh(db_perfil)
32     return db_perfil
33
34 # Deixei sem a função delete pois a intenção é ter perfis pré-definidos e usuários sempre linkados a estes perfis
```

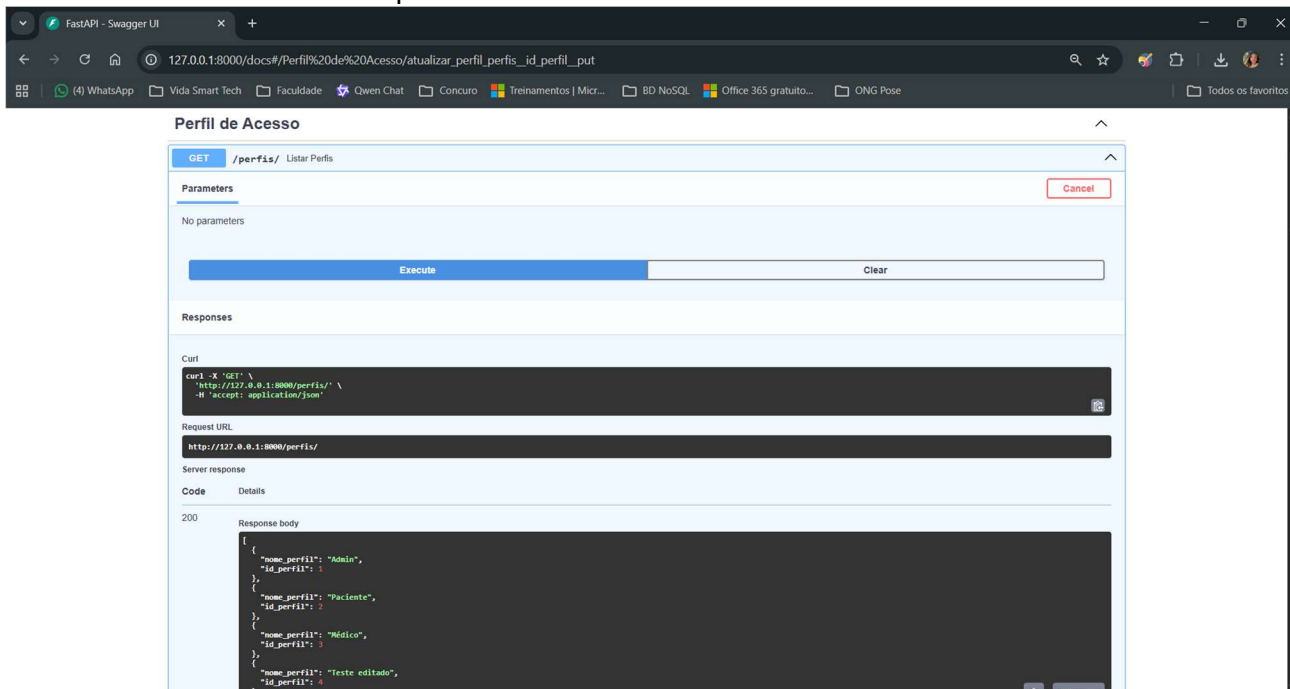
Prints Swagger:

Tela inicial:



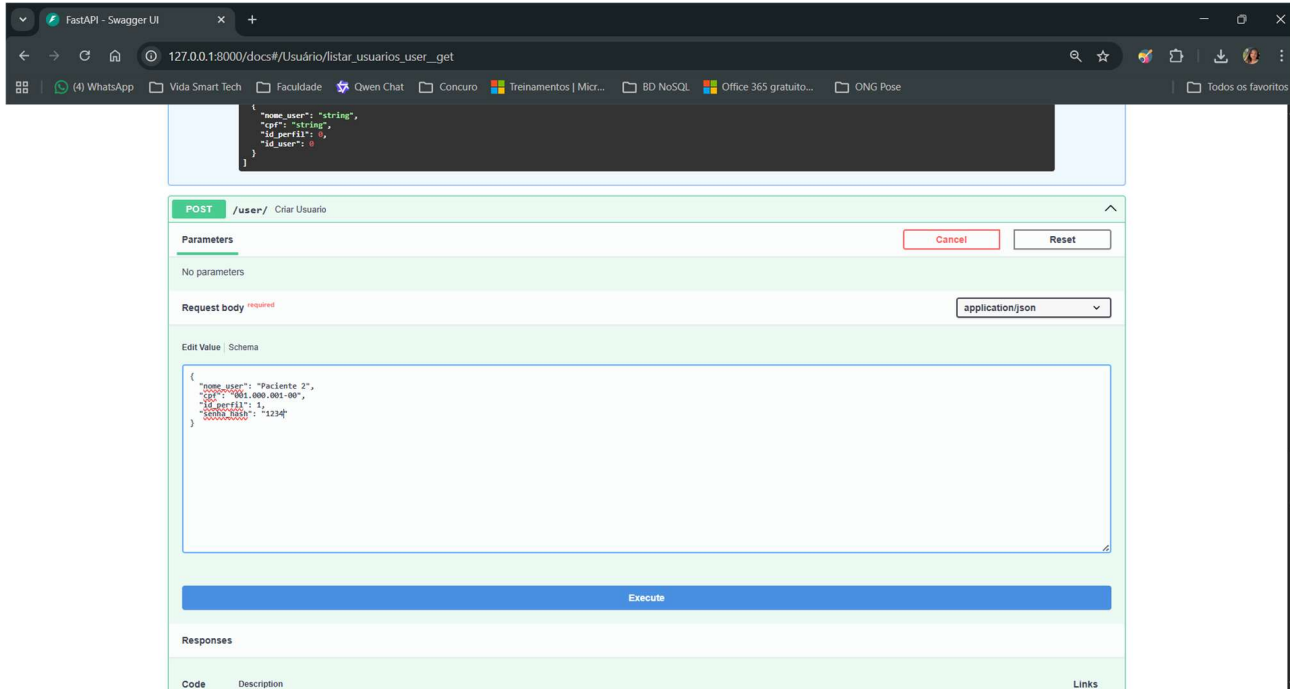
Perfil de acesso:

Esta sendo exibido os perfis criados

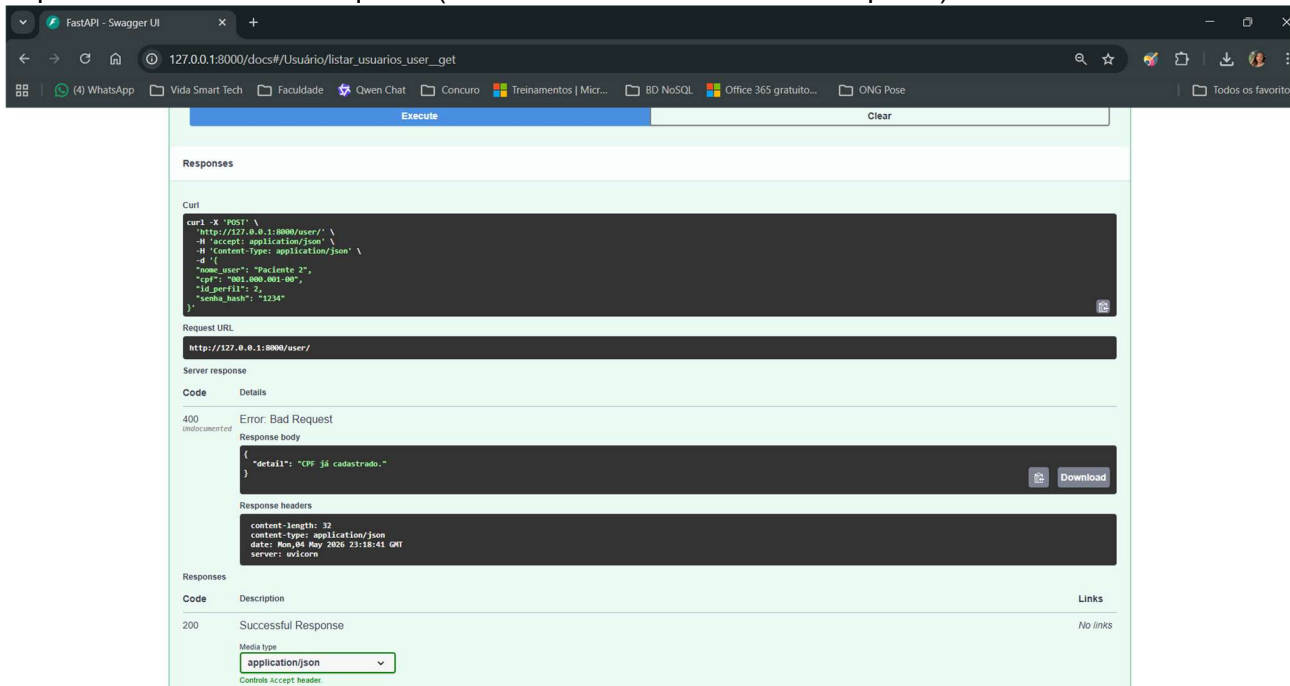


PLANO DE TESTES:

Cadastro de Novo Usuário: Verificação da criação de registros na tabela User com retorno de Status 200:



Validação de Duplicidade (CPF): Tentativa de cadastrar um usuário com CPF já existente, esperando o erro "Bad Request" (conforme evidenciado nos seus prints):



Atualização de Dados (Paciente): Teste da rota PUT /pacientes para alteração de campos específicos como data de nascimento:

The top screenshot shows the Swagger UI interface for the PUT /pacientes/{id_user} endpoint. The parameters section shows id_user as a required integer with a value of 2. The request body is set to application/json and contains the following JSON:

```
{  "data_nascimento": "01/01/2021"}
```

The bottom screenshot shows the execution results. The curl command is:

```
curl -X PUT \  "http://127.0.0.1:8000/pacientes/2" \  -H 'accept: application/json' \  -H 'content-type: application/json' \  -d '{  "data_nascimento": "01/01/2021"  }'
```

The request URL is http://127.0.0.1:8000/pacientes/2. The server response is 200 OK. The response body is:

```
{  "id_user": 2,  "data_nascimento": "01/01/2021",  "tipo_sanguine": "A",  "id_paciente": 1,  "user": {    "nome_user": "Paciente",    "cpf": "000.000.000-01",    "id_perfil": 1,    "id_user": 2  }}
```

The response headers are:

```
content-length: 362  content-type: application/json  date: Mon, 04 May 2026 23:38:51 GMT  server: nginx
```

The response is a successful response (200 OK) with the media type application/json.

Agendamento de consultas: Valida a capacidade do sistema de registrar um novo atendimento, vinculando um paciente a um médico (ambos já cadastrados) em uma data e hora específicas.

The first screenshot shows the Swagger UI interface for the endpoint `POST /consultas/ Agendar Consulta`. The request body is defined as `application/json` and contains the following JSON structure:

```
{  "id_paciente": 1,  "id_medico": 1,  "data_consulta": "01/05/2026",  "hora_consulta": "13h00",  "tipo_consulta": "Presencial",  "status": "Agendada"}
```

The second screenshot shows the same endpoint after a successful POST request. The response code is `200` and the response body is:

```
{  "id_paciente": 1,  "id_medico": 1,  "data_consulta": "01/05/2026",  "hora_consulta": "13h00",  "tipo_consulta": "Presencial",  "status": "Agendada",  "id_consulta": 6,  "prestador": null}
```

The response headers are:

```
content-length: 167
content-type: application/json
date: Mon, 04 May 2026 23:43:58 GMT
server: uvicorn
```

The response table shows a `200` status with the description `Successful Response`.

CONCLUSÃO:

O desenvolvimento do backend para o sistema **SGHSS** (Sistema de Gestão Hospitalar e de Serviços de Saúde) permitiu consolidar os conhecimentos adquiridos durante o curso, aplicando na prática conceitos de modelagem de dados, arquitetura de APIs com **FastAPI** e persistência com **SQLAlchemy**. O protótipo entregue cumpre os requisitos funcionais essenciais, garantindo a integração entre pacientes, médicos e agendamentos.

Entretanto, o processo de implementação revelou desafios técnicos significativos que impediram a conclusão do projeto exatamente conforme o planejado originalmente. Entre as principais dificuldades enfrentadas, destacam-se:

- **Manipulação de Tipagem:** Houve dificuldade na implementação de tipos complexos para datas e horários, resultando na decisão de mantê-los como variáveis do tipo *string* para garantir a funcionalidade imediata do sistema.
- **Restrições de Domínio:** Não foi possível restringir os campos de "tipo de consulta" estritamente aos valores "Presencial" ou "Telemedicina", permanecendo como campos de texto aberto.
- **Lógica de Busca em Endpoints:** Os métodos de atualização (PUT) foram implementados utilizando o ID (Chave Primária) como parâmetro de busca. Embora a intenção original fosse realizar essas operações através do Nome ou CPF para melhorar a experiência do usuário, limitações técnicas no desenvolvimento da lógica de filtragem impediram essa transição a tempo da entrega.

Apesar de não ter alcançado o nível de refinamento desejado em todos os módulos, o projeto atinge seu objetivo pedagógico ao apresentar uma estrutura de backend sólida, modularizada e funcional. As limitações encontradas ficam registradas como oportunidades de melhoria para futuras iterações do sistema, onde pretende-se aprimorar a validação de tipos e a flexibilidade das consultas na base de dados.